Interpreter

Interpreter · Evaluate arithmetic AST (2+3)*4 · Build with Waqar

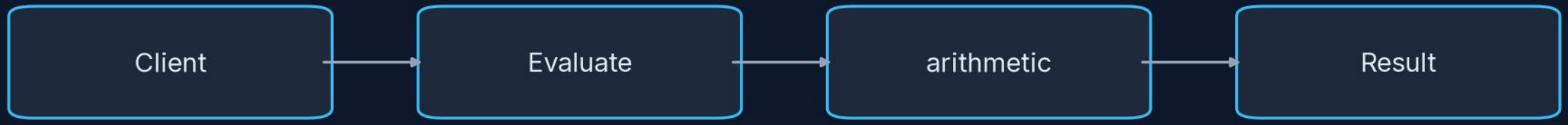
Today's Hook

- Problem: Evaluate arithmetic AST (2+3)*4
- Pattern: Interpreter — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Evaluate arithmetic AST (2+3)*4
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Interpreter: Evaluate arithmetic AST (2+3)*4

Real-World Scenario

- Running example: Evaluate arithmetic AST (2+3)*4
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.gof.behavioral.interpreter;

import com.buildwithwaqar.patterns.gof.behavioral.interpreter.*;
import java.util.Map;
/** Problem: evaluate a formula AST from a rules engine. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Interpreter problem: evaluate (2+3)*4 ===");
    Expression e = new MulExpr(new AddExpr(new NumberExpr(2), new NumberExpr(3)), new NumberExpr(4));
    System.out.println("value=" + e.evaluate(Map.of()));
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.gof.behavioral.interpreter;

import java.util.Map;

public sealed interface Expression permits NumberExpr, AddExpr, MulExpr {
    int evaluate(Map<String, Integer> ctx);
}

record NumberExpr(int value) implements Expression {
    public int evaluate(Map<String, Integer> ctx) { return value; }
}

record AddExpr(Expression left, Expression right) implements Expression {
    public int evaluate(Map<String, Integer> ctx) {
        return left.evaluate(ctx) + right.evaluate(ctx);
    }
}

record MulExpr(Expression left, Expression right) implements Expression {
    public int evaluate(Map<String, Integer> ctx) {
        return left.evaluate(ctx) * right.evaluate(ctx);
    }
```

Solution (2)

```
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.gof.behavioral.interpreter;

import com.buildwithwaqar.patterns.gof.behavioral.interpreter.*;
import java.util.Map;
/** Problem: evaluate a formula AST from a rules engine. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Interpreter problem: evaluate (2+3)*4 ===");
    Expression e = new MulExpr(new AddExpr(new NumberExpr(2), new NumberExpr(3)), new NumberExpr(4));
    System.out.println("value=" + e.evaluate(Map.of()));
  }
}
```

Walkthrough Output

```
=== Interpreter problem: evaluate (2+3)*4 ===
value=20
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Interpreter.

Related Patterns

- Contrast Interpreter with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Interpreter → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.